

交接文档：Agent 能力层与飞书集成

代码基线：main @ fd6ef94f (2026-08-06) 。

仓库 github.com/genuineknowledge/psi-agent。文中数字均按该基线实测。

0. 交接范围：两个维度

我负责的工作分两个维度，边界清晰：

维度一 —— Workspace 能力层 ([examples/haitun-workspace/](#))：工具、技能、系统提示词。这层决定「模型能做什么、知道什么」。

维度二 —— 飞书集成：从飞书长连接到 Session 的整条链路。

[src/psi_agent/channel/feishu/](#) (接入)、[src/psi_agent/gateway/_feishu_manager.py](#) (路由与多用户隔离)，以及能力层里的飞书专用工具与技能。

1. 先理解基础概念：Session

后面两个维度都绕不开 Session，所以先讲它。

Q1：什么是 session？

一个 Session = 一个独立的 agent 运行实例，是个真实的操作系统进程，对外暴露 HTTP/SSE 服务。它自带四样东西：

组成	说明
workspace	用户目录，相对文件 IO 的基准 (--workspace ，空则 <code>cwd</code>)
agent	Agent 包目录，工具/技能/ systems/ 从这里加载 (--agent ，空则同 <code>workspace</code>)
history	对话历史，落盘在 AppData 的 histories/
channel_socket	对外通信端点 (Unix socket / TCP / Windows 命名管道)

它做的事 ([SessionAgent.run\(\)](#) 的 agent loop)：

收到 channel 请求 → 惰性构建系统提示词 → 把 `history + tools` 发给 AI → 消费流式响应
→ 遇 `finish_reason="tool_calls"` 就执行工具 → 结果回填 history → 再发一轮直到 `stop`
(最多 `max_tool_rounds` 轮)

三个值得知道的设计：

1. **回合级原子性**：正常出口都 `commit()`（落盘 + 清快照），异常自动 `rollback()`。内存和磁盘只在同一检查点同步。
2. **同进程多 Session 隔离**：工具和提示词模块的加载用 `原名_session_id_文件 hash` 作为 module name，且走 `compile + exec` 而非 `importlib`，避免 bytecode 缓存串味。
3. ``runtime_context`` 是刻意窄用的：ContextVar 只由 `SessionAgent.run` 写入，只给 workspace 工具读（解析相对路径、拿当前会话 id）。框架核心一律用构造时的显式字段。别为了「方便全局拿一下」往里加东西 —— AppData 根、API key、provider 都明令禁止。

Q2：多 session 模式的用处是什么？

一句话：让互不相关的人和场景各有一份独立的上下文、记忆、工作目录，同时共享同一套能力层。

具体解决四个问题：

1.隐私隔离。

飞书场景是多租户的：A 的对话历史、文件、记忆绝不能被 B 看到。一人一个 Session 是最干净的实现 —— 隔离由进程边界保证，不靠代码里的 if 判断。

2.上下文连贯。

每个 Session 有自己的 history，所以机器人对每个人（或每个群）都有独立且连续的记忆，不会把张三的事说给李四。

3.并发。

Session 内是 `anyio.Lock` 串行（一个人的消息按序处理），Session 间天然并行。一个人的长任务不阻塞其他人。

4.故障隔离。

单个 Session 崩了或上下文压满了，不影响别人。代价是每个 Session 是一个进程，有内存开销，所以是**按需 spawn**（首次见到某人才建），不是预先起一堆。具体路由规则见 §3.2。

维度一：Workspace 能力层

2. 三层结构与调用机制

Q3: tools 与 skills 有什么区别？

这是能力层最核心的判断。

	Tool (工具)	Skill (技能)
本质	一个 Python 函数	一个 Markdown 文档 (<i>SKILL.md</i>)
给模型的形式	JSON Schema, 常驻上下文	索引里一行描述, 正文按需读
上下文成本	高 —— 整个函数签名 + 参数说明常驻	低 —— 只有一行摘要常驻
能做的事	真正执行代码、强制不变量、拦截危险操作	提供「怎么做」的知识、编排已有工具、调外部 CLI
改动生效	需重启 Session	改 Markdown 即生效

判据：

需要执行代码、需要护栏、需要维持不变量 → 工具。

只是「怎么做」的知识、多步 SOP、调外部 CLI → 技能。

为什么这个判断重要：

工具的成本是整个 schema 常驻上下文，向**所有会话**收税；技能的成本只有索引里的一行。

这层最稀缺的资源是上下文预算。**拿不准就先做技能。** *skills/skill-authoring-when/* 和 *skills/skill-authoring-how/* 就是写给这个判断的。

最典型的正面例子是 *feishu_api* (\$4.3)：几百个飞书端点本来可以做成几百个工具，最后压成「一个通用工具 + 技能里的端点表格」。

Q4: agent 如何调用这些 tools 与 skills？（重要性低）

两条完全不同的路径，这是理解能力层的关键。

Tools —— 走标准的 function calling 协议

1.加载: Session 启动时 `ToolRegistry.load(tools_dir)` 扫 `agent/tools/*.py`, 跳过 `_` 开头的文件。用 `compile + exec` 加载 (不用 `importlib`, 避免 `bytecode` 缓存)。

2.生成 Schema: `ToolFunction.from_callable()` 用 `inspect.signature()` 读函数签名 + `typing.get_type_hints()` 解析类型注解 + 解析 Google 风格 docstring 的 `Args:` 段, 自动合成 JSON Schema。

- 工具的 docstring 就是模型看到的说明书。改模型行为不用改代码, 改 docstring 就行。
- 支持的参数类型有限: `str / int / float / bool / list[X] / Literal / X | None`。别的会在加载时直接抛 `TypeError`。

3.调用: Session 把 schema 随 `history` 一起发给 AI → 模型返回 `tool_calls` → Session 按 index 拼接流式的 partial JSON → `registry.get(name)` 拿到函数 → `await` 执行 → 结果作为 tool message 追加进 `history` → 再发一轮。

4.收尾: `finish_reason="stop"` 时结束, `commit()` 落盘。

Skills —— 没有调用协议, 靠「读文件」

技能不是工具, 模型不能「调用」它。机制是:

1.`_build_skills_index()` 扫 `agent/skills/*/SKILL.md`, 抽出每个技能的名字和描述, 拼成 `<available_skills>` XML 塞进系统提示词。有 sha256 manifest 做快照缓存, 内容没变就不重新拼。

2.提示词里给的指令 (`SKILLS_HEADER_TEMPLATE`) 大意是: 扫一遍 `<available_skills>`, **如果有明显适用的, 用 `read` 工具读它的 `SKILL.md`, 然后照着做**; 多个适用就选最具体的; 都不明显适用就一个都别读; **一次最多一个**; 绝不臆造技能路径。

3.于是「调用技能」实际是模型自己决定发起一次 `read` 工具调用。技能正文只在真正需要时才进上下文 —— 这就是它成本低的原因。

两个推论: 技能之间可以互相引用 (都是文件路径); 技能里可以要求模型去调某几个工具, 所以「技能编排工具」是常见组合 —— 业务 SOP 类技能基本都这么写。

2.1 全局技能与就近覆盖

技能索引先扫全局目录，再扫 workspace 目录，同名则 workspace 覆盖全局（nearest wins，和 AGENTS.md 的查找约定一致）。

3. 系统提示词（systems/）

3.1 五个对外接口（Session 内核调用）

systems/system.py（1600+ 行）暴露给 Session 加载器的入口：

函数	作用	陷阱
system_prompt_builder()	建整个系统提示词	一个会话 只建一次 ，之后靠 checker 决定是否重建
system_prompt_rebuild_checker()	判断要不要重建	目前恒返 False —— 所以易变内容不能放进 system prompt
turn_context_builder()	每回合挂在本回合 user 消息 上的易变块	时钟、runtime、日历必须走这里
compact_history()	上下文压缩	见 §3.3
system_after_turn()	回合结束钩子， 当前未接线	AGENTS.md 明确写了「不要当死代码删」

另有可选的 system_before_turn()，接收本回合的临时 hook context；异常/非 dict 返回值降级成 {}，但不吞取消信号；schedule.* 回合跳过它。完整生命周期是：

system_before_turn → system_prompt_builder → AI/tool loop → system_after_turn。

最容易踩的坑：把时间、模型名、runtime 信息写进 system_prompt_builder。提示词只构建一次，checker 又恒 False，结果模型永远看到会话创建那一刻的时钟。正确做法是挂进 turn_context_builder。

3.2 提示词的组装来源

systems/prompt_sections.py 提供 section 构造函数（build_tooling_section、build_skills_section、build_workspace_section、build_runtime_line、build_model_identity_line），常量文本在 systems/prompt_constants.py。内容来自 workspace 根目录的几个 Markdown 基座文件：

- **SOUL.md** —— 人格/身份，始终注入
- **IDENTITY.md / USER.md** —— 身份与用户画像
- **BOOTSTRAP.md** —— 启动上下文文件
- **TOOLS.md** —— 工具使用总则
- **HEARTBEAT.md** —— 心跳轮行为
- **AGENTS.md** —— 本 workspace 的开发约定（**接手第一份必读**）

技能索引和工具名都是**自动扫**出来的（`_build_skills_index / _scan_tool_names`），不需要手工登记。注意这两条扫描管线和 `ToolRegistry` 是**两套**，历史上出过错配（某工具注册了但提示词里没露出，这会导致 agent 无法调用相应的工具）。

4. 工具层 (**tools/**)

4.1 规模与命名约定

- 目录共 136 个文件；**90 个是工具入口**（文件名不以 `_` 开头）。
- `_` 开头的是**实现层/helper**，不会被注册成工具。

工具入口保持薄（几十行签名 + docstring），真实现下沉到 `_xxx_impl.py`。这个范式有原因：**docstring 就是模型看到的 schema**，入口越薄 schema 越干净。改功能改 `_impl`，改模型行为改入口 docstring。

4.2 技能层规模

共 **138 个技能目录**。每个是一个目录 + **SKILL.md**，被自动扫进提示词索引。

4.3 「接口变数据」——上下文预算的核心决策

这是能力层最值得学的一次重构，也是 Q1 判据的最佳实例。

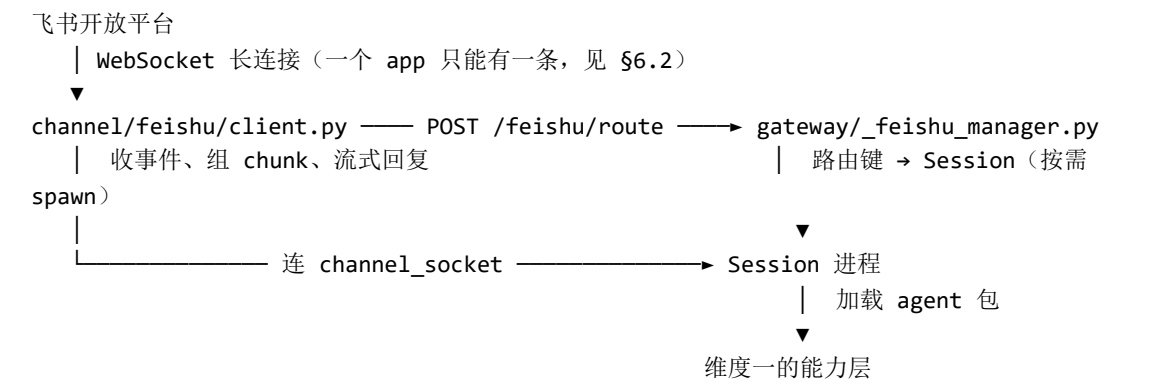
飞书工具一度占掉 **48.6%** 的上下文预算。PR #612 的做法：把不需要专用工具的端点从「一个工具一个 schema」改成**技能里的端点表格** + 一个通用 ``feishu_api`` 工具执行。结果工具数 319 → 248，飞书占比降到 **36.7%**。

判断标准写在 `feishu_api.py` 的 docstring 里，值得原样保留：**只有「形状容易搞错」的端点才配专用工具** —— 两步二进制上传、会静默丢数据的 sheet 坐标、必须先解析 id 才能删的 reaction。其余的只是 URI + JSON body，单独做成工具是净亏。

端点表格在 `skills/feishu-api/SKILL.md` (288 行)，覆盖 10 个域。表格解析器是 `_feishu_spec.py` (用 `_RULES_BLOCK` 正则提 ``rules 块) 。

维度二：飞书集成

5. 分层总览



关键设计：channel 只负责接飞书协议，不做任何路由决策。谁该进哪个 Session 完全由 Gateway 的 `FeishuManager` 决定。这条分工是刻意的，改动时不要让 channel 重新学会自己 spawn。

6. 接入层：src/psi_agent/channel/feishu/

Q5：agent 如何与飞书连接？（飞书 app 相关）

第一步：在飞书开放平台建一个企业自建应用，拿到 `app_id` 和 `app_secret`。这对凭证是整条链路的起点。

第二步：连接方式选「长连接」而非 `webhook`。这是本项目的关键选择——用飞书官方 SDK 的 `WebSocket` 长连接接收事件，好处是**不需要公网可访问的回调地址**，本地开发和内网部署都能直接收事件。代价见下面的铁律。

第三步：配置凭证。CLI 参数优先于环境变量：

配置	CLI	环境变量
App ID	<code>--app-id</code>	<code>PSI_FEISHU_APP_ID</code>
App Secret	<code>--app-secret</code>	<code>PSI_FEISHU_APP_SECRET</code>

两者都缺则启动时直接抛 `ValueError`，不会静默跑起来。

第四步：在开放平台订阅事件 + 申请权限（scope）。 权限是**按需申请**的白名单式 —— 要用哪个接口就申请哪个 scope，并且必须求各场景的**并集**，不要一次性申请全量。但可以测试时一次性申请全量的权限，减少一个个寻找对应权限的时间成本。

第五步：启动。 `ChannelFeishu` 是 CLI 入口的配置 dataclass，关键开关：

开关	默认	作用
<code>gateway_url</code>	None	设了才启用多用户路由；None 则全体共用一个 Session
<code>require_mention</code>	True	群里只在被 @ 时才回复；私聊不受影响
<code>respond_to_mention_all</code>	False	@all 默认 不 触发机器人
<code>respond_to_comments</code>	True	文档评论区 @ 机器人时自动回复
<code>allowed_user_ids</code>	None	白名单，None = 放开所有人

本地起完整链路用 `scripts/dev-feishu.ps1`（PR #608 加的 canonical 启动脚本）。

第六步（多租户场景）：让用户各自 OAuth 授权。 机器人默认用 tenant token 干活，但读写「某人自己的数据」需要该用户的 `user_access_token`。详见 §7。

6.1 文件职责

文件	职责
<code>client.py</code>	主体。WS 长连接、事件分发、组 chunk、流式回复、评论处理、审批事件
<code>_agent_events.py</code>	把 agent 包的 <code>channel_events/feishu/</code> 接到 <code>Session /events</code>
<code>_card_action.py</code>	卡片交互回调
<code>_card_store.py</code>	卡片状态存储
<code>__init__.py</code>	<code>ChannelFeishu</code> 配置 dataclass（CLI 入口）

6.2 铁律：一个 app 只能有一条 WebSocket

同一个飞书 app 只能有一条 WS 消费者。本机跑多个实例会互相抢事件，表现是消息随机丢 —— 这个现象很迷人，因为两个实例日志看起来都正常。

排查「连不上飞书」的顺序：WS 是否建起 → /ais 是否为空 → 网关端口是否错配 → 是否有重复通道。最常见真因是 state/latest.json 丢了 ais。

7. 路由层：gateway/_feishu_manager.py

这个文件是「飞书会话 → Session」的**唯一权威**，是 Q5（多 session）在飞书场景的落地。

会话类型	路由键	session_id	workspace
私聊（p2p/缺失）	裸 open_id	feishu-<open_id>（- → _）	<root>/<open_id>
群聊（group/topic 且 chat_id 非空）	chat:<chat_id>	feishu-chat-<chat_id>	<root>/chat-<chat_id>

五条设计要点，改动时别破坏：

- 1.私聊一人一个 Session，各自独立历史/workspace/记忆；群聊**整群共用一个**，机器人在群里因此有连贯上下文，且群与群、群与私聊互不串味。
- 2.`chat:` 前缀 + 私聊侧 ``→`` 转义是防隐私事故的：否则某人 open_id 恰为 chat-oc_x 时会和群 oc_x 撞成同一个 Session，陌生人共享上下文。
- 3.session_id 由路由键**确定性派生**，所以 _routes 只是内存缓存，重启后走 adopt 分支自愈，不需要持久化。
- 4.`chat\_id` 缺失时**退回按 open_id**，宁可隔离也不建 feishu-chat- 这种无主 Session。
- 5.Gateway 侧 --feishu-ai-id / --feishu-workspace-root 决定挂哪个 AI 和 workspace，channel 完全不关心。Gateway 不可达或路由失败时，channel **回退共享 `session\_socket`**——用户总能得到回复，只是不隔离。这是刻意的降级策略。

8. 多租户身份与 token 策略

- **tenant 优先**：策略下沉进 _invoke(prefer=tenant/user)。默认用机器人 token，仅在 tenant 确实被拒时回落到用户 token。
- **UAT 按 sender 分键**：每人各自授权后即以本人身份读写。全局搜索、memory、session socket 全按 open_id 隔离。
- **`identity` 参数显式选 user/bot**：创建「归属某人」的内容时必须指定，否则东西挂在机器人名下。

- **scope 按需申请**：白名单式，要求并集。

授权链路已做到**全链路非阻塞**——收码搬进了 `asyncio` 后台任务。历史坑：卡片 handler 才是真正的阻塞点，且 `anyio task group` 在那里不能用。一键授权卡的设计是发卡即收尾、只私聊、一次性。飞书中国区**没有 device flow**，所以走回调自动落地 + PKCE。

9. 飞书工具与技能（能力层里的飞书部分）

9.1 工具清单（按域）

19 个入口文件共暴露 **83 个工具函数**：

文件	函数数	覆盖
<code>feishu_message.py</code>	17	收发/撤回/已读/置顶/转发/卡片/图片下载
<code>feishu_doc.py</code>	11	文档创建、块级编辑、表格、嵌入 sheet
<code>feishu_auth.py</code>	8	OAuth 授权、UAT 管理、环境自检
<code>feishu_bitable.py</code>	7	多维表格 app/table/field/record
<code>feishu_chat.py</code>	6	建群、成员增删、群详情、列成员
<code>feishu_contact.py</code>	5	部门树、按名搜人、用户组
<code>feishu_wiki.py</code>	5	知识库节点与文档
<code>feishu_drive.py</code>	4	云盘上传、权限、评论
<code>feishu_sheet.py</code>	4	电子表格读写/格式
<code>feishu_approval.py</code>	3	审批定义查询、代提交、订阅
<code>feishu_calendar.py</code>	3	日程读取、逐人建日程
<code>feishu_auth_setup.py</code>	3	授权引导（只读自检类）
<code>feishu_chart.py</code>	2	matplotlib 渲图 → 文档图片块
<code>feishu_api.py</code>	1	通用端点调用 （见 §4.3）
其余	各 1	attendance / docs / todo_card / todo_card_tick

实现层集中在 `_feishu_impl.py`（**8385 行**，全仓最大单文件）、`_feishu_api_impl.py`、`_feishu_spec.py`、`_feishu_md.py`、`_feishu_auth_watch.py`、`_feishu_confirm.py`。

9.2 不可逆操作的确认码护栏

`_feishu_confirm.py` 实现了一条重要安全机制：解散群、员工离职、删部门、删用户组、删多维表格 table 这几个不可逆端点，**必须由真人给 6 位确认码**。首次调用不发请求，而是私聊把码发给 `user_key` 对应的人；模型拿不到码就做不了。一码一目标、15 分钟过期、一次性。

新增不可逆能力时请沿用这条，不要绕。 这也是「为什么某些端点必须是工具而非技能」的最好例证 —— 护栏只有工具能强制。

9.3 业务技能分组

- **平台能力类**：`feishu-api`（端点表格）、`feishu-message`、`feishu-chat`、`feishu-contact`、`feishu-drive`、`feishu-bitable`、`feishu-task`、`feishu-attendance`、`feishu-approval`、`feishu-charts`、`feishu-permission`
- **业务 SOP 类**：`feishu-admin-finance-assistant`、`feishu-leave-audit-board`、`feishu-reimbursement-audit-report`、`feishu-reimbursement-archive`、`feishu-attendance-payroll`、`feishu-resume-review`、`feishu-handbook-onboarding`、`feishu-mentor-feedback`、`feishu-work-handoff-delegate`、`feishu-blocker-routing`、`feishu-share-brief`、`feishu-self-service-agent`、`feishu-schedule-message`、`feishu-event-remind`、`feishu-todo-board-sync`、`feishu-todo-card`
- **合同法务**：`contract-law-source`、`contract-ledger`、`contract-review-sop`、`contract-legal-governance`（法条出处以国家法律法规数据库为准，禁编造）

10. 事件触发机制： `channel_events/feishu/`

agent 包里有 20 个事件定义目录，每个含 `EVENT.yaml` + `produce.py`（或 `map.py`）。

`_agent_events.py` 负责接到 `Session /events`。业务事件如 `attendance_review_needed`、`expense_submitted`、`blocker_raised`、`handoff_needed`、`task_overdue`。

10.1 为什么需要「事件」这层：从被动应答到主动触发

前面九节讲的能力都是**被动**的：用户说一句，agent 答一句。但真知管理的多数 SOP 不是这个形状 —— 「新员工入职了要发手册」「报销提交了要审」「任务逾期了要催」，这些事**没有人会主动来问**

agent。事件层解决的就是这个：把平台推送、表变更、工具写完状态这类**可观测信号**变成 agent 的一次输入，于是 agent 能在没人召唤时自己动起来。

判定一件事该不该做成触发器（这条判据在 `channel_events/README.md` 里，很值得沿用）：

- 存在可观测信号（平台推送 / 表变更 / 工具写完状态）→ **是**，做触发器。
- 纯对话入口、口语办业务、现场问答 → **不是**，走 Channel 或技能。
- 固定日历（发薪日、每周报表）→ 优先**定时任务**，不要做成 event。

换句话说，事件的本质是「**外界状态变了**」，定时任务的本质是「**时间到了**」。混用这两者是常见设计错误——把发薪日做成 event 就得自己造时钟，把入职做成 cron 就只能靠轮询猜。

10.2 目录即事件表：为什么没有中央注册表

Session **没有**业务 event catalog —— `channel_events/` 这个目录本身就是事件表。加一个事件 = 加一个目录 + 更新 README 里的表格，不需要改任何框架代码。

```
channel_events/feishu/<slug>/
  EVENT.yaml      # name, source, platform_event?, kind, filters?
  map.py          # kind=platform_map 必需: map_event(raw) -> list[dict]
  produce.py      # kind=synthetic 必需: async produce(ctx) -> None
```

`source` vs `event` 的区别要分清：`source` 是**管道品牌**（`feishu` / `haitun`，很少新增）；`event` 是管道里的**具体一件事**（经常新增）。一个 source 下挂很多 event。加信号默认只加 event；只有出现全新一类生产者才开新 source（还要改 Session 的 `KNOWN_SOURCES`）。**禁止 invent 表外名**——名字是契约，触发器靠它挂钩。

10.3 两种 kind：官方映射 vs 合成事件

这是理解事件层最关键的一刀。两种 kind 的**信号来源完全不同**，因此生命周期、热重载能力、失败模式都不一样：

	<code>kind: platform_map</code> (官方映射)	<code>kind: synthetic</code> (合成事件)
信号来自	飞书平台推送的官方事件	我们自己的代码（轮询表、查状态）
必需文件	<code>map.py</code> → <code>map_event(raw)</code>	<code>produce.py</code> → <code>async produce(ctx)</code>
运行形态	被动等 WS 投递，来一条跑一次	Channel 为每个 producer 起一条 常驻 task
改完是否要重启	不用 ，数秒内热重载生效	要重启 Channel
典型例子	<code>user_created</code> 、 <code>member_added</code> 、	<code>task_overdue</code> 、

	<code>identity_changed</code>	<code>expense_submitted、 handoff_needed</code>
--	-------------------------------	--

为什么 synthetic 不能热重载：一个正在运行的 producer task 无法安全替换（它可能正卡在 sleep 或半个 HTTP 请求里），所以热重载只覆盖 `platform_map`。这不是偷懒，是刻意的取舍 —— 写在 `_watch_channel_events` 的 docstring 里。

热重载为什么重要：它补的是「改了却没法验证」这个缺口。agent 自己写了新事件目录或修了 `map.py` 的字段路径后，下一次投递就能看到效果，而不需要它自己做不到的「重启 Channel」这个动作。

10.4 platform_map 的核心机制：包裹 fan-out

这是整个事件层**最反直觉、也最容易改坏**的一处，务必理解。

飞书 SDK 内部有个 `_processorMap`，键形如 `p2.im.message.receive_v1`，一个键只能挂一个处理器。问题在于：**机器人自己的回复路径已经占用了 `p2.im.message.receive_v1`**（经 `channel.on("message")` 注册）。

于是出现了一个极其迷惑的症状 —— 「**mapper 加载成功、`probe` 自查也正常，但线上永不触发**」。真因是：注册时看到 `p2` 键被占，就退而把 mapper 挂到空闲的 `p1.*` 键上，而 **WS 传输根本不用 `p1` 这套 schema**，于是 mapper 挂在了一个永远没有流量的键上。

正确解法是包裹，不是抢占（`_AgentEventFanout`）：

- 4. 键上没有处理器 → 直接装自己的，并打上 `_OURS_ATTR` 标记。
 - 5. 键上已有内置处理器 → 用 `_AgentEventFanout` **包住它**：`do()` 里**先跑内置**（保住用户可见的回复），再把同一条投递 fan-out 给各 mapper。
 - 6. `_is_ours()` 判定保证幂等 —— 热重载反复调用注册函数时，不会把自己的 fan-out 再包一层。
- 两个实现细节是硬要求：**`type()` 必须转发给内层** —— dispatcher 会在 `do()` 之前调它来选反序列化目标，返回别的类型会让内置处理器拿到错类型的对象；以及 fan-out 的异常**必须吞掉只记 warning** —— **绝不能让 mapper 的问题害得用户收不到回复**。这条优先级排序（内置回复 > 事件分发）是刻意的。

10.5 幂等：为什么 key 必须带 event_id

飞书会重试推送，所以同一件事可能投递多次。Session 侧靠 `idempotency_key` 去重，mapper 没填时由 `_forward_one` 兜底生成：`<event_name>:<delivery_id>:<index>`，其中 `delivery_id` 取 `header.event_id`（`P1` 取 `uuid`）。

历史坑：lark SDK 的事件模型没有 `dict()` 方法，早期代码被 `repr()` 兜底降级成了字符串，导致取不到 `event_id`、幂等键退化成常量，于是触发器只触发一次就再也不触发了（后续全被当重复投递丢掉）。修法是 `_raw_to_dict` 正确拆 SDK 模型。要点：**Session 的去重本身没错，错的是喂给它的 key。**

10.6 诊断：空结果的歧义与 filters

`map.py` 返回 `[]` 时，日志和「去重跳过」长得一模一样 —— 这是这层最大的可观测性陷阱，所以**写完必须自查：**

```
channel_event_check(action="list")
# 到底加载了哪些事件
channel_event_check(action="shape", platform_event="im.message.receive_v1")
# 字段真实在哪一层
channel_event_check(action="probe", event="feishu.chat.member_added")
# 用样例事件试跑自己的 map.py
```

`shape` 用**真实`lark\_channel` SDK 模型**造样例，所以它给的路径就是线上路径。最典型的坑：

`im.message.receive_v1` 的 `chat_id` 在 `event['message']['chat_id']`，**不在** `event['chat_id']`。

``filters: true` 的用途` (`EVENT.yaml`，默认 `false`)：只在「**大多数投递按设计就该返回`[]`**」时声明。

例如 `identity_changed` 订阅很宽的 `contact.user.updated_v3`，但组织里绝大多数变更是改头像/手机号，它一律丢弃。声明后**只有日志级别变化**：空结果记 `DEBUG` 而非 `WARNING`。

为什么这个开关值得存在：不声明的话每次有人改头像都刷一条「event dropped」警告，读者很快学会忽略它 —— **而这条诊断本来是用来抓「字段路径写错」的**，被噪声淹掉就白瞎了。反过来，只在畸形载荷时才返回 `[]` 的 mapper（如 `member_added` / `user_created`）**不要声明**：它们的空结果确实是故障，该报警。

10.7 投递终点与两条禁忌

mapper 产出的 `envelope` 经 `routing.open_id` 决定进哪个 Session (`resolve_core`)，最终落到 `ChannelCore.post_event`。缺省字段 (`schema_version` / `source` / `event` / `raw_event`) 由 `_forward_one` 从 `EVENT.yaml` 补齐，所以 mapper 只需关心业务字段。

- **processor 不能闭包捕获 def** —— 必须像 `_on_event` 那样用默认参数绑定 (`_platform_event=platform_event`)，否则循环里所有 processor 会共享最后一个值。
- **别让 agent 重启自己所在的 channel** —— 它会把自己弄死，且没有人能把它拉起来。

11. 后续优化方向

11.1 上下文预算：仍有一半空间可挖（高优先）

现状：飞书工具经 PR #612 从 48.6% 压到 **36.7%**，但这仍是单一最大占用方。可继续的方向：

- **把 #612 的「接口变数据」推广到非飞书域**。同样的判据适用于 GitHub、Discord、Notion 这些工具组——它们现在还是「一个端点一个工具」的老形态。
- **工具按需加载 (lazy schema)**。目前所有工具的 schema 无条件常驻。理想形态是按会话场景只装载相关工具组——飞书私聊场景根本用不到 `comfyui` / `discord_admin`。这个改动收益最大但动的是 `ToolRegistry`，属跨边界改动，需和 Session 负责人对齐。
- **度量先行**。用 `ToolRegistry.load` 实测占比，别靠数行数估。这条在 #612 已验证过一次：净行数完全不能反映上下文成本。

11.2 拆 `_feishu_impl.py`（高优先）

8385 行的单文件是最大的技术债。它现在混着消息、文档、表格、通讯录、审批等所有域的实现，后果是：改任何一域都要在这个文件里翻找、多人并行改必冲突、测试无法按域隔离。

建议按域拆成 `_feishu/{message,doc,sheet,bitable,contact,approval}.py`，保持 `_feishu_impl.py` 作为聚合入口 (re-export) 以免动到 19 个工具入口的 import。这是**纯机械重构**，适合作为接手后的**第一个练手任务**——既能强制读一遍全部实现，风险又低。

11.3 事件层：补齐 synthetic 的热重载（中优先）

现在 `platform_map` 能热重载而 `synthetic` 不能 (§10.3)，导致 agent 自己写的合成事件生产者改完无法生效——而它又不能重启自己所在的 Channel (§10.7)。这是个真实的能力缺口。

可行做法：给 producer task 加**协作式取消点**（每轮循环检查 fingerprint 是否变化，变了就主动退出让 runner 重起），而不是从外部强杀。需要约定 `produce(ctx)` 的写法规范，属于对 agent 作者的接口变更，要同步改 README。

11.4 提示词：把「只建一次」这个约束显式化（中优先）

`system_prompt_rebuild_checker()` 恒返 False 是个**隐式约束**：所有易变内容必须走 `turn_context_builder`，但这条规则只写在文档里，代码层面拦不住——新人往 `system_prompt_builder` 里塞个时间戳，测试全绿、线上悄悄错。

建议加一个测试：构建两次提示词（模拟不同时刻），断言输出**逐字节相同**。任何人塞进易变内容都会立刻红灯。这类「把踩过的坑变成测试」的改动性价比极高。

11.5 工具/技能一致性自检（中优先）

`_scan_tool_names`（提示词侧）和 `ToolRegistry`（执行侧）是**两套独立管线**，历史上出过错配——工具注册成功但提示词里没露出，模型压根不知道有这个工具，表现为「这功能明明做了却从来不用」。

建议加一致性断言：两侧工具名集合必须相等，不等就在启动时报错而非静默。同理适用于技能索引与 `skills/` 实际目录。

11.6 技能治理：138 个技能已到需要分层的规模（低优先但会持续恶化）

技能索引里每个技能占一行描述，138 行摘要本身已是可观的常驻成本，且**只会随业务增长**。另外扁平的 138 个目录让「有没有相似技能」这个判断越来越难——而 `skill-authoring-when` 要求创建前必须先 `list` 检查。

两个方向：一是**按域分组**（索引里先给分类，模型按需展开某一类），二是**做一次去重审计**（我怀疑业务 SOP 类里有可合并项，但没有实测数据，建议接手后先量再动）。

11.7 可观测性：给「静默失败」都配上诊断（低优先）

这层历史 bug 有个共同形状：**没报错，但事情没发生**。事件不触发、文件发不出去、工具没露出、飞书写入被静默丢弃——全都是「日志干净但行为错」。`channel_event_check` 是针对这类问题的正确解法（造真实样例、打印可读路径），值得把这个模式推广到其他易静默失败的环节，尤其是**飞书写入类工具的字段校验**（列名对不上被静默丢弃已经修过一次，但只覆盖了 bitable）。